

MODULE *protocol*

Version 2 *Zcash P2P* protocol specification, following the draft *ZIP* “Version 2 *Zcash P2P* Network Protocol” (*zcash/zips* $\neq$ 1344).

Models connection setup and block synchronization over the stream layer of *streams.tla*: the *init* handshake on the dedicated handshake stream, protocol version negotiation, the long-lived block announcement streams (including the rule that at most one announcement stream of a type may be open per direction and the sender’s option to replace a finished or reset stream), and headers-first synchronization over get-headers and get-blocks request streams, each carrying exactly one request and its response.

Blocks are heights on one linear chain: only the peer at the highest height extends it, and a peer that is behind downloads the heights it lacks.

Communication keeps the legacy model’s message consumption discipline: a peer decides only from its own records; the only place both peers’ records are written together is the transport itself (connection open/close, stream open, and the freeing of a stream slot once both sides have closed it).

Two boolean constants select how a receiver interprets the draft:

<i>StrictSingleton</i>	TRUE a second open announcement stream of the same type from a peer is a <i>PROTOCOL_ERROR</i> (literal reading of “Announcement Streams”) FALSE the receiver accepts it and drains both
<i>RefusePreHandshake</i>	TRUE announcement streams that arrive before the local handshake completes are refused with <i>REFUSED</i> (“Connection Handshake”): MAY refuse) FALSE they are buffered until it completes

Peers in *ByzantinePeers* complete an honest handshake and may then commit wire-level mischief: a second *init* record, a stream of unknown type, a record of unknown kind on the handshake stream, and a stream finished before its type byte. The receiver’s obligations differ per the draft: the first and last are connection errors, the middle two MUST be tolerated (forward compatibility). Ghost flags record genuine violations so that *CloseAccountable* can check no honest receiver ever fires *PROTOCOL_ERROR* without one, and *EventuallyPunished* that every violation ends the connection.

Under *StrictSingleton* the invariant *NoHonestProtocolError* is violated: two conformant peers disconnect because stream independence lets a replacement stream’s type byte arrive before the old stream’s *FIN* or reset.

See `documents/v2-modeling.md` for the full write-up.

EXTENDS *TLC*, *Naturals*, *Sequences*, *FiniteSets*, *streams*, *records*

CONSTANT <i>InitialPeers</i>	set of peers
CONSTANT <i>MaxBlock</i>	maximum block height (initial and mined)
CONSTANT <i>MaxRestarts</i>	finish/reset of own announcement streams, per peer pair
CONSTANT <i>MaxHeaders</i>	headers per get-headers response (draft: 160)
CONSTANT <i>MaxBlocksPerRequest</i>	hashes per get-blocks request (draft: 128)
CONSTANT <i>MinVersion</i>	minimum protocol version of the draft (not yet assigned)
CONSTANT <i>Versions</i>	protocol versions a peer may advertise
CONSTANT <i>StrictSingleton</i>	see module comment
CONSTANT <i>RefusePreHandshake</i>	see module comment
CONSTANT <i>ByzantinePeers</i>	peers that may misbehave after an honest handshake
CONSTANT <i>MaxMischief</i>	Byzantine actions available per peer pair

CONSTANT *PunishUnknownType* TRUE closes on unknown stream types (buggy)

VARIABLE *nodes*

$vars \triangleq \langle nodes \rangle$

See *README* for an explanation of symmetry reduction.

$Symmetry \triangleq Permutations(InitialPeers)$

For each initial peer construct a set of all other peers.

$OtherPeers \triangleq [n \in InitialPeers \mapsto InitialPeers \setminus \{n\}]$

$Min(a, b) \triangleq \text{IF } a < b \text{ THEN } a \text{ ELSE } b$

$Max(a, b) \triangleq \text{IF } a > b \text{ THEN } a \text{ ELSE } b$

$Init \triangleq$

$\exists blockset \in [InitialPeers \rightarrow (1 \dots MaxBlock)] :$

$\exists vset \in [InitialPeers \rightarrow Versions] :$

$nodes = [i \in InitialPeers \mapsto [$
 $conn \mapsto [j \in OtherPeers[i] \mapsto \text{"none"}],$
 $close \mapsto [j \in OtherPeers[i] \mapsto NoCode],$
 $streams \mapsto [j \in OtherPeers[i] \mapsto [sid \in StreamIds(i, j) \mapsto NullStream]],$
 $init_sent \mapsto [j \in OtherPeers[i] \mapsto FALSE],$
 $init_recvd \mapsto [j \in OtherPeers[i] \mapsto FALSE],$
 $version \mapsto [j \in OtherPeers[i] \mapsto 0],$
 $peer_tip \mapsto [j \in OtherPeers[i] \mapsto 0],$
 $announced \mapsto [j \in OtherPeers[i] \mapsto 0],$
 $restarts \mapsto [j \in OtherPeers[i] \mapsto 0],$
 $want \mapsto [j \in OtherPeers[i] \mapsto \langle \rangle],$
 $score \mapsto [j \in OtherPeers[i] \mapsto 0],$
 $mischief \mapsto [j \in OtherPeers[i] \mapsto 0],$
 $violated \mapsto [j \in OtherPeers[i] \mapsto FALSE],$
 $blocks \mapsto 1 \dots blockset[i],$
 $my_version \mapsto vset[i]$
 $]]$

— Helpers —

$Height(n) \triangleq Cardinality(nodes[n].blocks)$

$Connected(n, m) \triangleq nodes[n].conn[m] \in \{\text{"initiator"}, \text{"responder"}\}$

The handshake is complete for n once it has both sent and received an *init* record (draft: "Handshake Sequence", step 3).

$HandshakeComplete(n, m) \triangleq nodes[n].init_sent[m] \wedge nodes[n].init_recvd[m]$

$S(n, m, sid) \triangleq nodes[n].streams[m][sid]$

Slots n may use to open a new stream toward m (lowest free slot is taken).

$FreeSlots(n, m) \triangleq \{k \in 1 \dots MaxStreams : S(n, m, \langle n, k \rangle).status = \text{"none"}\}$

$FreeSlot(n, m) \triangleq \text{CHOOSE } k \in FreeSlots(n, m) : \forall j \in FreeSlots(n, m) : k \leq j$

Announcement streams of type t that n currently sends to m on.

$LiveAnn(n, m, t) \triangleq \{sid \in StreamIds(n, m) : \\ \wedge sid[1] = n \\ \wedge S(n, m, sid).status = \text{"open"} \\ \wedge S(n, m, sid).rtype = t \\ \wedge S(n, m, sid).out = \text{"open"}\}$

Request streams n has outstanding toward m .

$OpenReq(n, m) \triangleq \{sid \in StreamIds(n, m) : \\ \wedge sid[1] = n \\ \wedge S(n, m, sid).status = \text{"open"} \\ \wedge S(n, m, sid).rtype \in RequestTypes\}$

The sequence of heights $a \dots b$ (empty when $b < a$).

$Range(a, b) \triangleq [i \in 1 \dots (b - a + 1) \mapsto a + i - 1]$

$Contiguous(seq) \triangleq \forall i \in 1 \dots (Len(seq) - 1) : seq[i + 1] = seq[i] + 1$

Heights of seq above h .

$Above(seq, h) \triangleq SelectSeq(seq, \text{LAMBDA } x : x > h)$

Handshake streams n knows about on its connection with m .

$HandshakeStreams(n, m) \triangleq \{sid \in StreamIds(n, m) : \\ \wedge S(n, m, sid).status = \text{"open"} \\ \wedge S(n, m, sid).rtype = HandshakeType\}$

Transport: once both peers have closed a stream its slot is freed.

$Settle(ns, n, m, sid) \triangleq \\ \text{IF } \wedge ns[n].streams[m][sid].status = \text{"closed"} \\ \wedge ns[m].streams[n][sid].status = \text{"closed"} \\ \text{THEN } [ns \text{ EXCEPT } ![n].streams[m][sid] = NullStream, \\ ![m].streams[n][sid] = NullStream] \\ \text{ELSE } ns$

Transport: n closes the connection to m with an application error code.

Both peers observe the close; data in flight is lost.

$Close(ns, n, m, code) \triangleq \\ [ns \text{ EXCEPT } ![n].conn[m] = \text{"closed"}, \\ ![n].close[m] = code, \\ ![n].streams[m] = [sid \in StreamIds(n, m) \mapsto NullStream], \\ ![m].conn[n] = \text{"closed"}, \\ ![m].close[n] = code,$

$$![n].init_sent[m] = \text{TRUE}]$$

n consumes the type byte of a stream m opened. What happens depends on the type and on n 's own state:

0x00 from the responder, or a second 0x00 $\rightarrow$ *PROTOCOL_ERROR* ("Connection Handshake")
 any other stream before n 's handshake completed $\rightarrow$ refuse with *REFUSED*, or wait (*RefusePreHandshake*)
 announcement duplicating an open one of type $t \rightarrow$ *PROTOCOL_ERROR*, or accept (*StrictSingleton*)
 otherwise $\rightarrow$ record the type

$RecvTypeByte \triangleq$

$\exists n \in \text{InitialPeers} :$

$\exists m \in \text{OtherPeers}[n] :$

$\exists sid \in \text{StreamIds}(n, m) :$

LET $s \triangleq S(n, m, sid)$

IN

$\wedge \text{Connected}(n, m)$

$\wedge s.status = \text{"open"}$

$\wedge s.rtype = \text{"unknown"}$

$\wedge \text{Len}(s.inq) > 0$

$\wedge \text{IsTypeByte}(\text{Head}(s.inq))$

$\wedge \text{LET } t \triangleq \text{Head}(s.inq).t$

$\text{accept} \triangleq [\text{nodes EXCEPT } ![n].streams[m][sid].rtype = t,$
 $![n].streams[m][sid].inq = \text{Tail}(@)]$

$\text{legalHs} \triangleq \text{nodes}[n].conn[m] = \text{"responder"} \wedge \text{HandshakeStreams}(n, m) = \{\}$

$\text{dup} \triangleq \exists o \in \text{StreamIds}(n, m) :$

$\wedge o \neq sid$

$\wedge o[1] = m$

$\wedge S(n, m, o).status = \text{"open"}$

$\wedge S(n, m, o).rtype = t$

IN

$\vee \wedge t \notin \text{StreamTypes}$

Unknown stream type: refuse with

UNSUPPORTED_STREAM_TYPE; the draft says MUST NOT

close or penalize — new stream types deploy

without version gating. *PunishUnknownType* is the

forbidden reading.

$\wedge \vee \wedge \text{PunishUnknownType}$

$\wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"PROTOCOL_ERROR"})$

$\vee \wedge \neg \text{PunishUnknownType}$

$\wedge \text{nodes}' = \text{Settle}([\text{nodes EXCEPT}$

$![n].streams[m][sid] = \text{ClosedStream},$

$![m].streams[n][sid] = \text{Refuse}(@, t, \text{"UNSUPPORTED_STREAM_TYPE"})],$

$n, m, sid)$

$\vee \wedge t \in \text{StreamTypes}$

$\wedge \vee \wedge t = \text{HandshakeType}$

$\wedge \vee \wedge \text{legalHs}$

$$\begin{aligned}
& \wedge nodes' = accept \\
& \vee \wedge \neg legalHs \\
& \quad \wedge nodes' = Close(nodes, n, m, "PROTOCOL_ERROR") \\
& \vee \wedge t \in AnnTypes \cup RequestTypes \\
& \quad \wedge \vee \wedge \neg HandshakeComplete(n, m) \\
& \quad \quad \wedge RefusePreHandshake \\
& \quad \quad \wedge nodes' = Settle([nodes \text{ EXCEPT} \\
& \quad \quad \quad ! [n].streams[m][sid] = ClosedStream, \\
& \quad \quad \quad ! [m].streams[n][sid] = Refuse(@, t, "REFUSED")], \\
& \quad \quad \quad n, m, sid) \\
& \vee \wedge HandshakeComplete(n, m) \\
& \quad \wedge \vee \wedge t \in AnnTypes \wedge StrictSingleton \wedge dup \\
& \quad \quad \wedge nodes' = Close(nodes, n, m, "PROTOCOL_ERROR") \\
& \quad \quad \vee \wedge \neg(t \in AnnTypes \wedge StrictSingleton \wedge dup) \\
& \quad \quad \wedge nodes' = accept
\end{aligned}$$

n consumes m 's *init* record from the handshake stream (draft: "Handshake Validation"):

second <i>init</i> record	$\rightarrow PROTOCOL_ERROR$
version below <i>MinVersion</i>	$\rightarrow OBSOLETE$
otherwise	$\rightarrow$ handshake received; negotiated <i>version</i> = <i>min</i>

$RecvInit \triangleq$

$$\begin{aligned}
& \exists n \in InitialPeers : \\
& \quad \exists m \in OtherPeers[n] : \\
& \quad \quad \exists sid \in HandshakeStreams(n, m) : \\
& \quad \quad \quad LET s \triangleq S(n, m, sid) \\
& \quad \quad \quad IN \\
& \quad \quad \quad \wedge Connected(n, m) \\
& \quad \quad \quad \wedge Len(s.inq) > 0 \\
& \quad \quad \quad \wedge Head(s.inq).kind = "init" \\
& \quad \quad \quad \wedge LET msg \triangleq Head(s.inq) \\
& \quad \quad \quad \quad IN \\
& \quad \quad \quad \quad \vee \wedge nodes[n].init_rcvd[m] \\
& \quad \quad \quad \quad \quad \wedge nodes' = Close(nodes, n, m, "PROTOCOL_ERROR") \\
& \quad \quad \quad \quad \vee \wedge \neg nodes[n].init_rcvd[m] \\
& \quad \quad \quad \quad \quad \wedge msg.version < MinVersion \\
& \quad \quad \quad \quad \quad \wedge nodes' = Close(nodes, n, m, "OBSOLETE") \\
& \quad \quad \quad \quad \vee \wedge \neg nodes[n].init_rcvd[m] \\
& \quad \quad \quad \quad \quad \wedge msg.version \geq MinVersion \\
& \quad \quad \quad \quad \wedge nodes' = [nodes \text{ EXCEPT} \\
& \quad \quad \quad \quad \quad \quad ! [n].streams[m][sid].inq = Tail(@), \\
& \quad \quad \quad \quad \quad \quad ! [n].init_rcvd[m] = TRUE, \\
& \quad \quad \quad \quad \quad \quad ! [n].version[m] = Min(nodes[n].my_version, msg.version), \\
& \quad \quad \quad \quad \quad \quad ! [n].peer_tip[m] = msg.start_height]
\end{aligned}$$

n ignores a record of unknown kind on the handshake stream: “a node MUST ignore handshake-stream records whose kind it does not recognize”
(draft: “Connection Handshake”).

$RecvUnknownHandshakeRecord \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\exists sid \in HandshakeStreams(n, m) :$
 $LET\ s \triangleq S(n, m, sid)$
 IN
 $\wedge Connected(n, m)$
 $\wedge Len(s.inq) > 0$
 $\wedge s.inq[1].kind \notin \{“init”, “type”, “fin”\}$
 $\wedge nodes' = [nodes\ EXCEPT\ ![n].streams[m][sid].inq = Tail(@)]$

— Byzantine actions —

A Byzantine peer completes an honest handshake and then misbehaves at the wire level, within a $MaxMischief$ budget. Ghost bookkeeping: the HONEST side’s violated flag records genuine violations(the first and last are; the tolerated two are not), so accountability is checkable.

$ByzCan(n, m) \triangleq$
 $\wedge n \in ByzantinePeers$
 $\wedge Connected(n, m)$
 $\wedge HandshakeComplete(n, m)$
 $\wedge nodes[n].mischief[m] < MaxMischief$

A second init record on the handshake stream | a violation the receiver MUST answer with $PROTOCOL_ERROR$ (draft : “Handshake Validation”).

$ByzSecondInit \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\exists sid \in HandshakeStreams(n, m) :$
 $\wedge ByzCan(n, m)$
 $\wedge nodes' = [nodes\ EXCEPT$
 $\quad ![m].streams[n][sid] = PutData(@, MakeInit(nodes[n].my_version, Height(n))),$
 $\quad ![n].mischief[m] = @ + 1,$
 $\quad ![m].violated[n] = TRUE]$

A record of unknown kind on the handshake stream | NOT a violation : future revisions may define new kinds and the receiver MUST ignore them.

$ByzUnknownHandshakeRecord \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\exists sid \in HandshakeStreams(n, m) :$

$$\begin{aligned}
& \wedge \text{ByzCan}(n, m) \\
& \wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad ! [m].\text{streams}[n][\text{sid}] = \text{PutData}(@, [\text{kind} \mapsto \text{"junk"}]), \\
& \quad ! [n].\text{mischief}[m] = @ + 1]
\end{aligned}$$

A stream of a type the receiver does not recognize | NOT a violation :
the receiver refuses it with `UNSUPPORTED_STREAM_TYPE` and moves on
(draft : "Stream Types").

$$\begin{aligned}
& \text{ByzUnknownStreamType} \triangleq \\
& \quad \exists n \in \text{InitialPeers} : \\
& \quad \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \quad \wedge \text{ByzCan}(n, m) \\
& \quad \quad \quad \wedge \text{FreeSlots}(n, m) \neq \{\} \\
& \quad \quad \quad \wedge \text{LET } \text{sid} \triangleq \langle n, \text{FreeSlot}(n, m) \rangle \\
& \quad \quad \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PeerStream}(\text{"0xEE"}, \langle \rangle), \\
& \quad \quad \quad \quad ! [n].\text{mischief}[m] = @ + 1]
\end{aligned}$$

A handshake stream opened by a peer that already has one(or is the
responder) | a violation the receiver *MUST* answer with `PROTOCOL_ERROR`
(draft : "Connection Handshake").

$$\begin{aligned}
& \text{ByzSecondHandshakeStream} \triangleq \\
& \quad \exists n \in \text{InitialPeers} : \\
& \quad \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \quad \wedge \text{ByzCan}(n, m) \\
& \quad \quad \quad \wedge \text{FreeSlots}(n, m) \neq \{\} \\
& \quad \quad \quad \wedge \text{LET } \text{sid} \triangleq \langle n, \text{FreeSlot}(n, m) \rangle \\
& \quad \quad \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PeerStream}(\text{HandshakeType}, \langle \rangle), \\
& \quad \quad \quad \quad ! [n].\text{mischief}[m] = @ + 1, \\
& \quad \quad \quad \quad ! [m].\text{violated}[n] = \text{TRUE}]
\end{aligned}$$

A stream finished before a complete type byte | a violation the receiver
MUST answer with `PROTOCOL_ERROR`(draft : "Stream Types").

$$\begin{aligned}
& \text{ByzEarlyFin} \triangleq \\
& \quad \exists n \in \text{InitialPeers} : \\
& \quad \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \quad \wedge \text{ByzCan}(n, m) \\
& \quad \quad \quad \wedge \text{FreeSlots}(n, m) \neq \{\} \\
& \quad \quad \quad \wedge \text{LET } \text{sid} \triangleq \langle n, \text{FreeSlot}(n, m) \rangle \\
& \quad \quad \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = [\text{NullStream} \text{ EXCEPT } !.\text{status} = \text{"open"}, \\
& \quad \quad \quad \quad \quad \quad \quad !.\text{inq} = \langle \text{FIN} \rangle],
\end{aligned}$$

$$\begin{aligned} ![n].mischief[m] &= @ + 1, \\ ![m].violated[n] &= \text{TRUE} \end{aligned}$$

— Announcement streams —

After its handshake completes, n opens one announcement stream of each type it has none open for (draft: “Announcement Streams”). This is also how a replacement is opened after a finish, reset, or refusal. A reset can overtake announcements still in flight, so the sender forgets what it announced and re-announces its tip on the new stream.

$\text{OpenAnnouncementStream} \triangleq$

$$\begin{aligned} &\exists n \in \text{InitialPeers} : \\ &\quad \exists m \in \text{OtherPeers}[n] : \\ &\quad \quad \exists t \in \text{AnnTypes} : \\ &\quad \quad \quad \wedge \text{Connected}(n, m) \\ &\quad \quad \quad \wedge \text{HandshakeComplete}(n, m) \\ &\quad \quad \quad \wedge \text{LiveAnn}(n, m, t) = \{\} \\ &\quad \quad \quad \wedge \text{FreeSlots}(n, m) \neq \{\} \\ &\quad \quad \quad \wedge \text{LET } sid \triangleq \langle n, \text{FreeSlot}(n, m) \rangle \\ &\quad \quad \quad \text{IN } nodes' = [nodes \text{ EXCEPT} \\ &\quad \quad \quad \quad ![n].streams[m][sid] = \text{OpenerStream}(t), \\ &\quad \quad \quad \quad ![m].streams[n][sid] = \text{PeerStream}(t, \langle \rangle), \\ &\quad \quad \quad \quad ![n].announced[m] = 0] \end{aligned}$$

n announces its current tip to m when it has grown since the last announcement (draft: “Block Announcements”, header announcement).

$\text{SendAnnouncement} \triangleq$

$$\begin{aligned} &\exists n \in \text{InitialPeers} : \\ &\quad \exists m \in \text{OtherPeers}[n] : \\ &\quad \quad \exists sid \in \text{LiveAnn}(n, m, \text{BlockAnnType}) : \\ &\quad \quad \quad \wedge nodes[n].announced[m] < \text{Height}(n) \\ &\quad \quad \quad \wedge nodes' = [nodes \text{ EXCEPT} \\ &\quad \quad \quad \quad ![m].streams[n][sid] = \text{PutData}(@, \text{MakeHeaderAnnouncement}(\text{Height}(n))), \\ &\quad \quad \quad \quad ![n].announced[m] = \text{Height}(n)] \end{aligned}$$

n consumes a header announcement from m . Announcements are only processed once n ’s handshake is complete (draft: “Connection Handshake”).

$\text{RecvAnnouncement} \triangleq$

$$\begin{aligned} &\exists n \in \text{InitialPeers} : \\ &\quad \exists m \in \text{OtherPeers}[n] : \\ &\quad \quad \exists sid \in \text{StreamIds}(n, m) : \\ &\quad \quad \quad \text{LET } s \triangleq S(n, m, sid) \\ &\quad \quad \text{IN} \\ &\quad \quad \quad \wedge \text{Connected}(n, m) \end{aligned}$$

$$\begin{aligned}
& \wedge \text{HandshakeComplete}(n, m) \\
& \wedge s.\text{status} = \text{"open"} \\
& \wedge s.\text{rtype} \in \text{AnnTypes} \\
& \wedge \text{Len}(s.\text{inq}) > 0 \\
& \wedge \text{Head}(s.\text{inq}).\text{kind} = \text{"header"} \\
& \wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad ! [n].\text{streams}[m][\text{sid}].\text{inq} = \text{Tail}(@), \\
& \quad ! [n].\text{peer_tip}[m] = \text{Max}(@, \text{Head}(s.\text{inq}).\text{height})]
\end{aligned}$$

n finishes one of its announcement streams. The draft allows a sender to open a replacement afterwards; why it finished (backpressure, internal restart) is not modeled. Bounded by MaxRestarts to keep liveness meaningful.

$\text{FinishAnnouncementStream} \triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists \text{sid} \in \text{LiveAnn}(n, m, \text{BlockAnnType}) : \\
& \quad \quad \quad \wedge \text{nodes}[n].\text{restarts}[m] < \text{MaxRestarts} \\
& \quad \quad \quad \wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PutData}(@, \text{FIN}), \\
& \quad \quad \quad \quad ! [n].\text{restarts}[m] = @ + 1], \\
& \quad \quad \quad n, m, \text{sid})
\end{aligned}$$

Same as $\text{FinishAnnouncementStream}$ but via RESET_STREAM , which can overtake records still in flight on that stream.

$\text{ResetAnnouncementStream} \triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists \text{sid} \in \text{LiveAnn}(n, m, \text{BlockAnnType}) : \\
& \quad \quad \quad \wedge \text{nodes}[n].\text{restarts}[m] < \text{MaxRestarts} \\
& \quad \quad \quad \wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PutReset}(@, \text{"INTERNAL_ERROR"}), \\
& \quad \quad \quad \quad ! [n].\text{restarts}[m] = @ + 1], \\
& \quad \quad \quad n, m, \text{sid})
\end{aligned}$$

— Request streams: headers-first synchronization —

n is behind m (from m 's init or announcements), has nothing queued to download and no request outstanding: it asks m for headers after its own tip (draft: "Headers-First Synchronization", step 1). The request and the FIN of n 's sending direction are written together.

$\text{SendGetHeaders} \triangleq$

$$\exists n \in \text{InitialPeers} :$$

$$\begin{aligned}
& \exists m \in \text{OtherPeers}[n] : \\
& \quad \wedge \text{Connected}(n, m) \\
& \quad \wedge \text{HandshakeComplete}(n, m) \\
& \quad \wedge \text{nodes}[n].\text{peer_tip}[m] > \text{Height}(n) \\
& \quad \wedge \text{nodes}[n].\text{want}[m] = \langle \rangle \\
& \quad \wedge \text{OpenReq}(n, m) = \{\} \\
& \quad \wedge \text{FreeSlots}(n, m) \neq \{\} \\
& \quad \wedge \text{LET } \text{sid} \triangleq \langle n, \text{FreeSlot}(n, m) \rangle \\
& \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{RequesterStream}(\text{GetHeadersType}), \\
& \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PeerStream}(\text{GetHeadersType}, \\
& \quad \quad \quad \quad \langle \text{MakeGetHeaders}(\text{Height}(n)), \text{FIN} \rangle)]
\end{aligned}$$

n serves a get-headers request from m : the headers after the locator, at most MaxHeaders of them, then FIN . Serving may start as soon as the request is complete, before m 's FIN has been consumed (draft: "Request Streams").

$$\begin{aligned}
\text{ServeGetHeaders} & \triangleq \\
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists \text{sid} \in \text{StreamIds}(n, m) : \\
& \quad \quad \quad \text{LET } s \triangleq S(n, m, \text{sid}) \\
& \quad \quad \quad \text{IN} \\
& \quad \quad \quad \wedge \text{Connected}(n, m) \\
& \quad \quad \quad \wedge \text{HandshakeComplete}(n, m) \\
& \quad \quad \quad \wedge s.\text{status} = \text{"open"} \\
& \quad \quad \quad \wedge s.\text{rtype} = \text{GetHeadersType} \\
& \quad \quad \quad \wedge s.\text{out} = \text{"open"} \\
& \quad \quad \quad \wedge \text{Len}(s.\text{inq}) > 0 \\
& \quad \quad \quad \wedge \text{Head}(s.\text{inq}).\text{kind} = \text{"get-headers"} \\
& \quad \quad \quad \wedge \text{LET } \text{loc} \triangleq \text{Head}(s.\text{inq}).\text{locator} \\
& \quad \quad \quad \quad \text{hdrs} \triangleq \text{Range}(\text{loc} + 1, \text{Min}(\text{loc} + \text{MaxHeaders}, \text{Height}(n))) \\
& \quad \quad \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{Collapse}([\text{@} \text{ EXCEPT } !.\text{inq} = \text{Tail}(\text{@}), \\
& \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad !.\text{out} = \text{"finished"}]), \\
& \quad \quad \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PutData}(\text{PutData}(\text{@}, \text{MakeHeaders}(\text{hdrs})), \text{FIN})]
\end{aligned}$$

n consumes a headers response from m (draft: "get-headers"):
more than MaxHeaders , or not contiguous $\rightarrow$ discard, misbehavior penalty
otherwise $\rightarrow$ queue the heights n lacks

$$\begin{aligned}
\text{RecvHeaders} & \triangleq \\
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists \text{sid} \in \text{OpenReq}(n, m) : \\
& \quad \quad \quad \text{LET } s \triangleq S(n, m, \text{sid})
\end{aligned}$$

IN
 $\wedge \text{Connected}(n, m)$
 $\wedge s.\text{rtype} = \text{GetHeadersType}$
 $\wedge \text{Len}(s.\text{inq}) > 0$
 $\wedge \text{Head}(s.\text{inq}).\text{kind} = \text{"headers"}$
 $\wedge \text{LET } \text{hdrs} \triangleq \text{Head}(s.\text{inq}).\text{heights}$
 IN
 $\vee \wedge \text{Len}(\text{hdrs}) \leq \text{MaxHeaders} \wedge \text{Contiguous}(\text{hdrs})$
 $\wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT}$
 $\quad ! [n].\text{streams}[m][\text{sid}].\text{inq} = \text{Tail}(@),$
 $\quad ! [n].\text{want}[m] = \text{Above}(\text{hdrs}, \text{Height}(n))]$
 $\vee \wedge \neg(\text{Len}(\text{hdrs}) \leq \text{MaxHeaders} \wedge \text{Contiguous}(\text{hdrs}))$
 $\wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT}$
 $\quad ! [n].\text{streams}[m][\text{sid}].\text{inq} = \text{Tail}(@),$
 $\quad ! [n].\text{score}[m] = @ + 20]$

n requests the next batch of blocks it wants from m , at most
 $\text{MaxBlocksPerRequest}$ of them (draft: "get-blocks").

$\text{SendGetBlocks} \triangleq$
 $\exists n \in \text{InitialPeers} :$
 $\exists m \in \text{OtherPeers}[n] :$
 $\wedge \text{Connected}(n, m)$
 $\wedge \text{nodes}[n].\text{want}[m] \neq \langle \rangle$
 $\wedge \text{OpenReq}(n, m) = \{\}$
 $\wedge \text{FreeSlots}(n, m) \neq \{\}$
 $\wedge \text{LET } \text{sid} \triangleq \langle n, \text{FreeSlot}(n, m) \rangle$
 $\quad \text{batch} \triangleq \text{SubSeq}(\text{nodes}[n].\text{want}[m], 1,$
 $\quad \quad \text{Min}(\text{MaxBlocksPerRequest}, \text{Len}(\text{nodes}[n].\text{want}[m])))$
 IN $\text{nodes}' = [\text{nodes} \text{ EXCEPT}$
 $\quad ! [n].\text{streams}[m][\text{sid}] = \text{RequesterStream}(\text{GetBlocksType}),$
 $\quad ! [m].\text{streams}[n][\text{sid}] = \text{PeerStream}(\text{GetBlocksType},$
 $\quad \quad \langle \text{MakeGetBlocks}(\text{batch}), \text{FIN} \rangle)]$

n serves a get-blocks request from m . It may finish after any complete
 entry, delivering fewer blocks than requested (draft: "get-blocks"); the
 requester re-requests the remainder.

$\text{ServeGetBlocks} \triangleq$
 $\exists n \in \text{InitialPeers} :$
 $\exists m \in \text{OtherPeers}[n] :$
 $\exists \text{sid} \in \text{StreamIds}(n, m) :$
 $\quad \text{LET } s \triangleq S(n, m, \text{sid})$
 IN
 $\wedge \text{Connected}(n, m)$
 $\wedge \text{HandshakeComplete}(n, m)$
 $\wedge s.\text{status} = \text{"open"}$

n consumes delivered blocks from m and extends its chain with them.

$$! [n].want[m] = Above(nodes[n].want[m], Cardinality(blocks))$$

LET $s \triangleq S(n, m, sid)$

$$\begin{aligned}
&![n].streams[m][sid] = ClosedStream, \\
&![m].streams[n][sid] = PutReset(@, s.stop)], \\
&n, m, sid)
\end{aligned}$$

— Chain growth —

The peer at the chain tip finds a new block, giving it something to announce. Only the highest peer extends the chain, which keeps the model a single linear chain that the others catch up with.

$MineBlock \triangleq$

$$\begin{aligned}
&\exists n \in InitialPeers : \\
&\quad \wedge Height(n) < MaxBlock \\
&\quad \wedge \forall m \in OtherPeers[n] : Height(m) \leq Height(n) \\
&\quad \wedge nodes' = [nodes \text{ EXCEPT } ![n].blocks = @ \cup \{Height(n) + 1\}]
\end{aligned}$$

$Next \triangleq$

- $\vee Connect$
- $\vee OpenHandshakeStream$
- $\vee SendInitResponder$
- $\vee RecvTypeByte$
- $\vee RecvInit$
- $\vee RecvUnknownHandshakeRecord$
- $\vee ByzSecondInit$
- $\vee ByzUnknownHandshakeRecord$
- $\vee ByzUnknownStreamType$
- $\vee ByzSecondHandshakeStream$
- $\vee ByzEarlyFin$
- $\vee OpenAnnouncementStream$
- $\vee SendAnnouncement$
- $\vee RecvAnnouncement$
- $\vee FinishAnnouncementStream$
- $\vee ResetAnnouncementStream$
- $\vee SendGetHeaders$
- $\vee ServeGetHeaders$
- $\vee RecvHeaders$
- $\vee SendGetBlocks$
- $\vee ServeGetBlocks$
- $\vee RecvBlocks$
- $\vee RecvFin$
- $\vee RecvReset$
- $\vee RecvStop$
- $\vee MineBlock$

Weak fairness on *Next*: as long as any action is enabled, one is taken.
Mining and restarts are bounded, so every behaviour quiesces and every
enabled receive eventually happens. The one loop that is not bounded is a
responder refusing pre-handshake announcement streams while the sender
keeps reopening them; fairness on *RecvInit* guarantees the responder
eventually processes the *init* record that ends that loop.

$$\begin{aligned}
Spec &\triangleq \\
&Init \\
&\wedge \Box [Next]_{vars} \\
&\wedge WF_{vars}(Next) \\
&\wedge WF_{vars}(RecvInit)
\end{aligned}$$

— Liveness —

Every connection eventually completes its handshake (or has been closed).

$$\begin{aligned}
HandshakeCompletes &\triangleq \\
&\Diamond \Box \forall n \in InitialPeers : \forall m \in OtherPeers[n] : \\
&\quad nodes[n].conn[m] = \text{"closed"} \vee HandshakeComplete(n, m)
\end{aligned}$$

Every peer eventually learns every connected peer's final tip.

$$\begin{aligned}
AnnouncementsFlow &\triangleq \\
&\Diamond \Box \forall n \in InitialPeers : \forall m \in OtherPeers[n] : \\
&\quad nodes[n].conn[m] = \text{"closed"} \vee nodes[n].peer_tip[m] = Height(m)
\end{aligned}$$

Eventually all peers hold the same chain.

$$\begin{aligned}
EventualConsensus &\triangleq \\
&\Diamond \Box \forall i, j \in InitialPeers : nodes[i].blocks = nodes[j].blocks
\end{aligned}$$

— Safety invariants —

$$\begin{aligned}
TypeOK &\triangleq \\
&\forall n \in InitialPeers : \forall m \in OtherPeers[n] : \\
&\quad \wedge nodes[n].conn[m] \in \{\text{"none"}, \text{"initiator"}, \text{"responder"}, \text{"closed"}\} \\
&\quad \wedge nodes[n].close[m] \in ErrorCodes \cup \{NoCode\} \\
&\quad \wedge nodes[n].version[m] \in Versions \cup \{0\} \\
&\quad \wedge nodes[n].restarts[m] \in 0 \dots MaxRestarts \\
&\quad \wedge nodes[n].score[m] \in Nat \\
&\quad \wedge nodes[n].mischief[m] \in 0 \dots MaxMischief \\
&\quad \wedge \forall sid \in StreamIds(n, m) : \\
&\quad \quad \wedge S(n, m, sid).status \in \{\text{"none"}, \text{"open"}, \text{"closed"}\} \\
&\quad \quad \wedge S(n, m, sid).rtype \in StreamTypes \cup \{\text{"unknown"}\} \\
&\quad \quad \wedge S(n, m, sid).out \in \{\text{"open"}, \text{"finished"}, \text{"reset"}, \text{"na"}\}
\end{aligned}$$

A stream slot is free on one side exactly when it is free on the other.

$SlotsConsistent \triangleq$

$\forall n \in InitialPeers : \forall m \in OtherPeers[n] : \forall sid \in StreamIds(n, m) :$
 $(S(n, m, sid).status = \text{"none"}) = (S(m, n, sid).status = \text{"none"})$

At most one handshake stream per connection, opened by the initiator
(draft: "Connection Handshake").

$OneHandshakeStream \triangleq$

$\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $\wedge Cardinality(HandshakeStreams(n, m)) \leq 1$
 $\wedge \forall sid \in HandshakeStreams(n, m) :$
IF $sid[1] = n$ THEN $nodes[n].conn[m] = \text{"initiator"}$
ELSE $nodes[m].conn[n] = \text{"initiator"}$

A peer opens no stream before sending its *init* record, and no announcement
stream before its handshake completes (draft: "Connection Handshake").

$HandshakeBeforeStreams \triangleq$

$\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $\wedge \neg nodes[n].init_sent[m] \Rightarrow$
 $\forall k \in 1 \dots MaxStreams : S(n, m, \langle n, k \rangle).status = \text{"none"}$
 $\wedge \neg HandshakeComplete(n, m) \Rightarrow$
 $\forall k \in 1 \dots MaxStreams : S(n, m, \langle n, k \rangle).rtype \notin AnnTypes$

Once n has received m 's *init*, the negotiated version is the minimum of the
two advertised versions (draft: "Protocol Versioning").

$NegotiatedVersionIsMin \triangleq$

$\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $HandshakeComplete(n, m) \Rightarrow$
 $nodes[n].version[m] = Min(nodes[n].my_version, nodes[m].my_version)$

A sender never has two announcement streams of one type open toward a peer
(draft: "Announcement Streams").

$SenderSingleton \triangleq$

$\forall n \in InitialPeers : \forall m \in OtherPeers[n] : \forall t \in AnnTypes :$
 $Cardinality(LiveAnn(n, m, t)) \leq 1$

OBSOLETE is only ever used against a peer that advertised an old version.

$CloseJustified \triangleq$

$\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $nodes[n].close[m] = \text{"OBSOLETE"} \Rightarrow$
 $\vee nodes[n].my_version < MinVersion$
 $\vee nodes[m].my_version < MinVersion$

Blocks are always a contiguous chain from genesis.

$BlocksContiguous \triangleq$

$\forall n \in InitialPeers : nodes[n].blocks = 1 \dots Height(n)$

Request streams are only opened after the handshake completes, one at a

time per peer (draft: “Connection Handshake”; *ZIP* – 204 in-transit limit).

$RequestsAfterHandshake \triangleq$

$$\begin{aligned} & \forall n \in InitialPeers : \forall m \in OtherPeers[n] : \\ & \quad \wedge OpenReq(n, m) \neq \{\} \Rightarrow HandshakeComplete(n, m) \\ & \quad \wedge Cardinality(OpenReq(n, m)) \leq 1 \end{aligned}$$

Responses in flight respect the draft’s bounds: at most *MaxHeaders* contiguous headers, at most *MaxBlocksPerRequest* hashes per get-blocks.

$ResponsesBounded \triangleq$

$$\begin{aligned} & \forall n \in InitialPeers : \forall m \in OtherPeers[n] : \forall sid \in StreamIds(n, m) : \\ & \quad \forall i \in 1 \dots Len(S(n, m, sid).ing) : \\ & \quad \quad LET x \triangleq S(n, m, sid).ing[i] \\ & \quad \quad IN \quad \wedge x.kind = \text{“headers”} \Rightarrow Len(x.heights) \leq MaxHeaders \wedge Contiguous(x.heights) \\ & \quad \quad \quad \wedge x.kind = \text{“get-blocks”} \Rightarrow Len(x.heights) \leq MaxBlocksPerRequest \\ & \quad \quad \quad \wedge x.kind = \text{“blocks”} \Rightarrow Len(x.heights) \leq MaxBlocksPerRequest \end{aligned}$$

Honest peers never earn a misbehavior penalty.

$NoHonestPenalty \triangleq$

$$\forall n \in InitialPeers : \forall m \in OtherPeers[n] : nodes[n].score[m] = 0$$

No interleaving of conformant behaviour ends in a protocol error. Checked in configurations with no *Byzantine* peers, where any close other than

OBSOLETE is a disagreement between honest peers about the draft’s rules.

$NoHonestProtocolError \triangleq$

$$\begin{aligned} & \forall n \in InitialPeers : \forall m \in OtherPeers[n] : \\ & \quad nodes[n].close[m] \in \{NoCode, \text{“OBSOLETE”}\} \end{aligned}$$

Accountability: an honest receiver fires *PROTOCOL_ERROR* only after a genuine violation by the peer. Tolerated mischief (unknown stream types, unknown handshake records) never sets the ghost flag, so a receiver that punished it would violate this invariant.

$CloseAccountable \triangleq$

$$\begin{aligned} & \forall n \in InitialPeers \setminus ByzantinePeers : \forall m \in OtherPeers[n] : \\ & \quad nodes[n].close[m] = \text{“PROTOCOL_ERROR”} \Rightarrow nodes[n].violated[m] \end{aligned}$$

Every genuine violation eventually ends the connection.

$EventuallyPunished \triangleq$

$$\begin{aligned} & \forall n \in InitialPeers : \forall m \in OtherPeers[n] : \\ & \quad \Box(nodes[n].violated[m] \Rightarrow \Diamond(nodes[n].conn[m] = \text{“closed”})) \end{aligned}$$